

Primitive Data Type

Primitive is the most fundamental data type usable in the Programming language. There are eight primitive data types: **Boolean**, **byte**, **character**, **short**, **int**, **long**, **float**, and **double**. In a Programming language, these data types serve as the foundation for data manipulation.

All basic data types are built-in into the majority of programming languages. Furthermore, many languages provide a set of composite data types. Primitive data types may or may not have a one-to-one correspondence with objects in the computer's memory, depending on the language and its implementation. However, operations on basic primitive data types are typically thought to be the fastest language constructs.

The actual number of primitive data types available is determined by the programming language that is used. Strings, for instance, are a composite but built-in data type in C#, even as they are integrated to a primitive data type that is both basic and built-in in advanced accents of BASIC and JavaScript.

Considering the Java Programming language, the primitive data structures include integers, floats, characters, and pointers. In general, there are 8 data types. They are as follows:

- Boolean data type
- byte data type
- char data type
- short data type
- int data type
- long data type
- float data type
- double data type

Boolean Data Type

A Boolean data type comprises a single bit of information that can only store true or false values. True or false conditions are tracked using this data type, and Boolean data types are also used to store the result of various conditions. Let's write a small program and see how it works.

```
1. class booleanDataType{
2. public static void main(String args[]){
3. // Setting the values for boolean data type
4. boolean Java = true;
5. boolean Python = false;
6. System.out.println(Java); // Output will be true
7. System.out.println(Python); // Output will be false
8. }
```

9. }

Byte Data Type

The byte data type is an illustration of a primitive data type. It is a signed two's complement integer of 8 bits, and it stores whole numbers ranging from -128 to 127. A byte data type is useful for saving large amounts of memory. Let's write a small program and see how it works.

```
1. class ByteExample {
2.     public static void main(String[] args) {
3.         byte n, a;
4.         n = 127;
5.         a=177;
6.         System.out.println(n); // prints 127
7.         System.out.println(a); // throws an error because it cannot store more than 127 bits
8.     }
9. }
```

Char Data Type

A single character is stored in this data type, and the character must be enclosed in single quotes, such as 'E' or 'e'. You can also use ASCII values to display specific characters. Let's look at a simple example to see how it works.

Short Data Type

A short data type is larger than a byte but smaller than an integer, and it saves values ranging from -32,768 to 32767. This data type's default size is 2 bytes. Let's look at an example to understand the short data type better.

Int Data Type

This data type is capable of storing whole numbers ranging from -2147483648 to 2147483647. When creating variables with numeric values, int is generally the preferred data type.

Long Data Type

This data type is a two's complement 64-bit integer. A long data type's default size is 64 bits, and its value ranges from -263 to 263-1.

Float Data Type

It would be best to use a floating-point type when you need a number with a decimal, such as 8.88 or 3.14515.

This data type supports fractional numbers ranging from 3.4×10^{-38} to 3.4×10^{38} . It is important to note that the value should end with an "f." Let's look at a specific example to understand this data type better.

Double Data Type

The double data type can store fractional numbers from 1.7×10^{-308} to 1.7×10^{308} . Note that you should end the value with a "d":

Pointer Definition in C

What are Pointers?

A pointer is a type of variable in the C language that is used to **store the address of another variable**. The pointers can be of any data type, such as int, float, char, string, array, or function. In C language, pointers are represented using the **asterisk symbol (*)**. The size of the pointers depends on the architecture. For example, the size of the pointer will be 4 bytes for a 32-bit machine or 8 bytes for a 64-bit machine.

Features of pointers in C language

- Pointers are used to **save the memory** of the computers.
- It provides **direct access to the memory address** and thus saves time.
- Pointers are **used with data structures** such as linked lists.
- **Dynamic allocation** as well as de-allocation of memory is done using pointers.

Syntax for declaring a pointer in C language:

- **data_type * variable_name;**

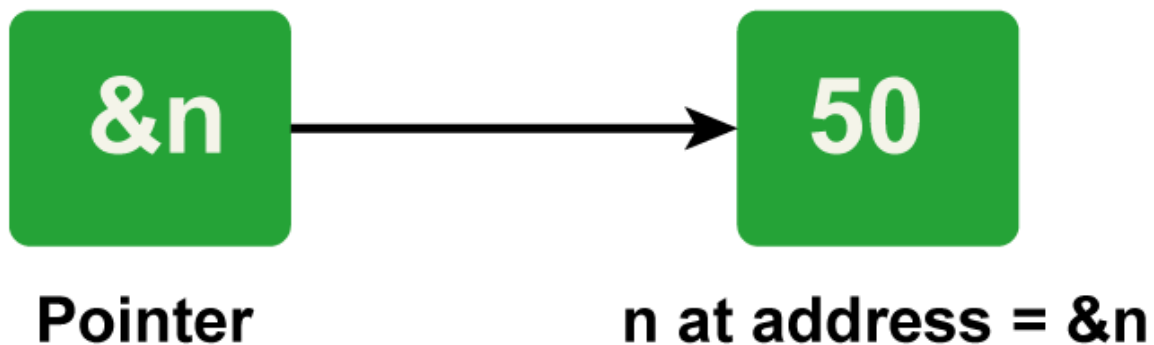
1. `int * ptr; // here a pointer is created with name ptr and data type as int.`
2. `char * ptr; // a pointer is created with name ptr and data type as char.`

Array of pointers

Array of pointers in C language is the collection of pointers and is represented as follows:

- **data_type * variable_name[size];**

1. `int * ptr[5];` // array of pointers with name ptr is created having size 5 and data type as int.
2. `char * ptr[10];` array of pointers with name ptr is created having size 10 and data type as char.

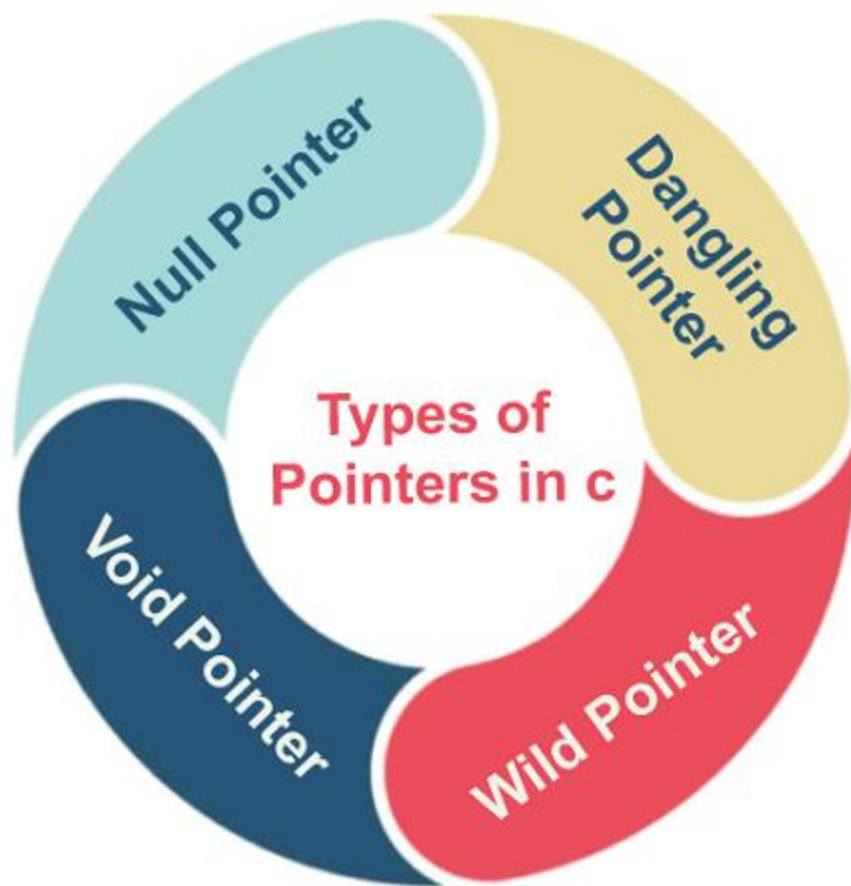


1. `#include <stdio.h>`
2. `Int main() {`
3. `int marks = 75;`
4. `printf(“%p”, &marks);` // & is used to access the memory address of the variable ?marks?.
5. `Return 0;`
6. `}`

Output:

```
0x77fee2365f099 // memory address of the variable 'marks'. (Note: Output varies from
```

Types of Pointers in C language



NULL Pointers:

Null Pointers are the pointers that have **no value assigned to it**. NULL pointers do not point to any function or object. It means that the pointer has not been assigned with any valid memory address.

Wild Pointers:

When you don't initialize anything to the pointer, the pointer is called as a wild pointer. This type of pointer may lead to the crashing of the program because the pointer will assign to a random memory address.

Void Pointers:

Void pointers are the pointers that do not have any specific data type. It is created by using the void keyword.

```
1. #include<stdio.h>
2. int main(){
3. void *ptr = NULL; //void pointer
4. return 0;
5. }
```

Dangling Pointers:

When you de-allocate the memory of a pointer using the free() function, then that pointer is known as dangling pointer.

Structure

A structure is a composite data type that defines a grouped list of variables that are to be placed under one name in a block of memory. It allows different variables to be accessed by using a single pointer to the structure.

Syntax

```
1. struct structure_name
2. {
3.     data_type member1;
4.     data_type member2;
5.     .
6.     .
7.     data_type member;
8. };
```

Advantages

- It can hold variables of different data types.
- We can create objects containing different types of attributes.
- It allows us to re-use the data layout across programs.
- It is used to implement other data structures like linked lists, stacks, queues, trees, graphs etc.

Program

Array in Data Structure

In this article, we will discuss the array in data structure. Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

In C programming, they are the derived data types that can store the primitive type of data such as int, char, double, float, etc. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define a different variable for the marks in different subjects. Instead, we can define an array that can store the marks in each subject at the contiguous memory locations.

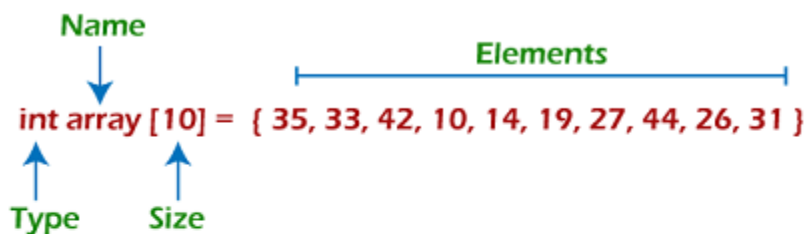
Properties of array

There are some of the properties of an array that are listed as follows -

- Each element in an array is of the same data type and carries the same size that is 4 bytes.
- Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Representation of an array

We can represent an array in various ways in different programming languages. As an illustration, let's see the declaration of array in C language -



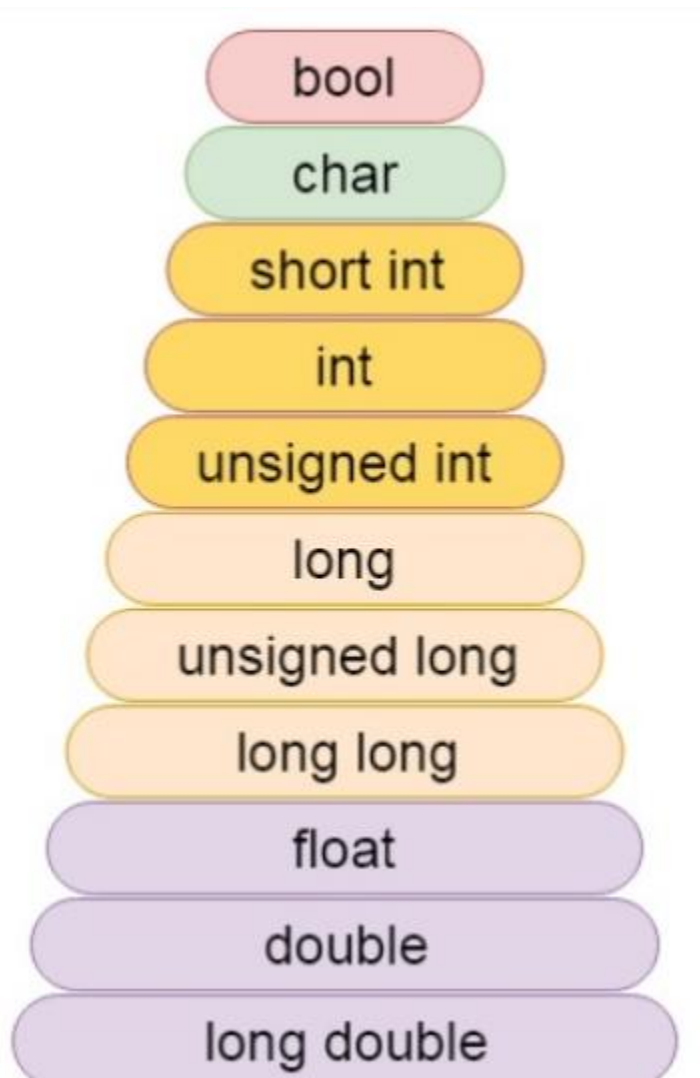
As per the above illustration, there are some of the following important points -

- Index starts with 0.
- The array's length is 10, which means we can store 10 elements.
- Each element in the array can be accessed via its index.

What is coercion in programming languages?

Many programming languages support the conversion of a value into another of a different data type. This kind of type conversions can be implicitly or explicitly made. Implicit conversion, which is also called coercion, is automatically done.

The Argument Coercion is one technique by which the compiler can implicitly convert the arguments from one type to another type. It follows argument promotion rule. If one argument is lower datatype, that can be converted into higher datatypes, but the reverse is not true.



Coercion in Java

Many programming languages, including **Java**, allow **conversion** of a data type to another data type as a convenience to the programmer. The kind of conversion may be **implicit** or **explicit**. The **implicit conversion** is done automatically by JVM but explicit conversion is done by the programmer.

What is Coercion?

In Java, the mechanism of converting one type of object to another object of a different type with similar content is known as coercion. In other words, we can say that coercion is the process of converting one data type to another data type. In a more specific way, **implicit conversion** is called **coercion**. It is also known as **type coercion**.

It occurs because the datum (information) is stored as one data type but its context requires a different data type. For example, the constant (sum) 20 is an integer but its context requires a double value.

1. **int** sum = 20;
2. **double** avg = sum/5;

Example of Coercion

Conversion of **String to Integer**, **Long to Double** would be coercion.

Another example, if we want to coerce the StringBuffer to an Integer, in such a case the following series of coercions will execute.

1. Object -> String -> Long -> Integer

Consider the following code statements. In the third statement, the value of the count is automatically converted to double before executing the division (/) operator.

Consider the following code statements. In the third statement, the value of the count is automatically converted to double before executing the division (/) operator.

1. **double** avg, sum;
2. **int** count;
3. ...
4. avg = sum / count;

There is a hidden machine operation performed on data at the execution time of the program.

- Fetches the value of the sum variable.
- Fetches the value of the count variable.
- Converts the value of the count variable to a floating-point.
- Performs floating-point division.
- Stores the result into memory allocated to avg variable.

Numeric Data Coercion

The following figure shows type conversions between Java's numeric data types.

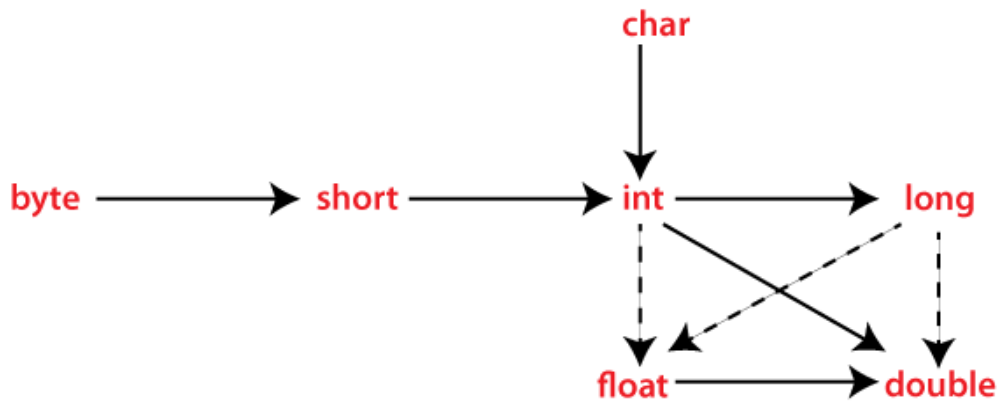


Fig: Coercion in Java

Java will automatically coerce data in the direction of the arrows, either solid or dashed. For example:

Coercion	Results
<code>int a = 'A';</code>	<code>a = 65</code>
<code>long b = 'A';</code>	<code>b = 65</code>
<code>double c = 'A';</code>	<code>c = 65.0</code>

Difference Between Type Casting and Type Coercion

The conversion of **primitive data type** into another data type is known as type casting or type conversion. There are two ways to cast the primitive data type, widening, and narrowing. It is also known as implicit type casting because it is done automatically.

Casting is the process by which we treat an object type as another type while coercing is the process of converting one object to another.

TYPE CHECKING RULES usually have the form

```

if two type expressions are equivalent
then return a given type
else return type_error
  
```

KEY IDEAS. The central issue is then that we have to define when two given type expressions are equivalent.

- The main difficulty arises from the fact that most modern languages allow the naming of user-defined types.
- For instance, in C and C++ this is achieved by the `typedef` statement.
- When checking equivalence of named types, we have two possibilities.

Name equivalence.

Treat named types as basic types. Therefore two type expressions are *name equivalent* if and only if they are identical, that is if they can be represented by the same syntax tree, with the same labels.

Structural equivalence.

Replace the named types by their definitions and recursively check the substituted trees.

STRUCTURAL EQUIVALENCE. If type expressions are built from basic types and constructors (without type names, that is in our example, when using products instead of records), structural equivalence of types can be decided easily.

- For instance, to check whether the constructed types `array(n1, T1)` and `array(n2, T2)` are equivalent
 - we can check that the integer values `n1` and `n2` are equal and recursively check that `T1` and `T2` are equivalent,
 - or we can be less restrictive and check only that `T1` and `T2` are equivalent.
- Compilers use representations for type expressions (trees or dags) that allow type equivalence to be tested quickly.

RECURSIVE TYPES. In PASCAL a *linked list* is usually defined as follows.

```
type link = ^ cell;
cell = record
    info: type;
    next: link;
end;
```

The corresponding type graph has a cycle. So to decide structural equivalence of two types represented by graphs PASCAL compilers put a *mark* on each visited node (in order not to visit a node twice). In C, a *linked list* is usually defined as follows.

```
struct cell {
    int info;
    struct cell *next;
};
```

To avoid cyclic graphs, C compilers

- require type names to be declared before they are used, except for pointers to records.
- use structural equivalence except for records for which they use name equivalence.

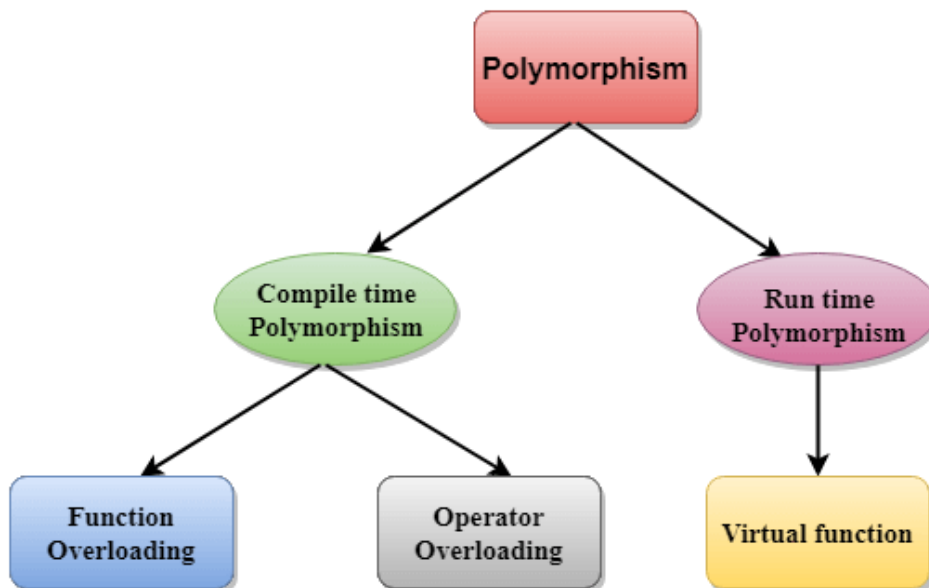
C++ Polymorphism

The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word. In object-oriented programming, we use 3 main concepts: inheritance, encapsulation, and polymorphism.

Real Life Example Of Polymorphism

Let's consider a real-life example of polymorphism. A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market. Here, a single person is behaving differently according to the situations.

There are two types of polymorphism in C++:



- **Compile time polymorphism:** The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Now, let's consider the case where function name and prototype is same.

```

1. class A                                // base class declaration.
2. {
3.     int a;
4.     public:
5.     void display()
6.     {
7.         cout<< "Class A ";
8.     }
9. };
10. class B : public A                    // derived class declaration.
11. {
12.     int b;
13.     public:
14.     void display()
15.     {
16.         cout<<"Class B";
17.     }
18. };

```

In the above case, the prototype of display() function is the same in both the **base and derived class**. Therefore, the static binding cannot be applied. It would be great if the appropriate function is selected at the run time. This is known as **run time polymorphism**.

- **Run time polymorphism:** Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.

It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

C++ Runtime Polymorphism Example

Let's see a simple example of run time polymorphism in C++.

// an example without the virtual keyword.

```

1. #include <iostream>
2. using namespace std;
3. class Animal {
4.     public:
5.     void eat(){
6.         cout<<"Eating...";
7.     }
8. };
9. class Dog: public Animal
10. {
11.     public:
12.     void eat()
13.     {         cout<<"Eating bread...";
14.     }
15. };
16. int main(void) {
17.     Dog d = Dog();
18.     d.eat();
19.     return 0;
20. }
```

Output:

```
Eating bread...
```

Runtime Polymorphism with Data Members

Runtime Polymorphism can be achieved by data members in C++. Let's see an example where we are accessing the field by reference variable which refers to the instance of derived class.

```
1. #include <iostream>
2. using namespace std;
3. class Animal {                      // base class declaration.
4.     public:
5.         string color = "Black";
6. };
7. class Dog: public Animal            // inheriting Animal class.
8. {
9.     public:
10.        string color = "Grey";
11. };
12. int main(void) {
13.     Animal d= Dog();
14.     cout<<d.color;
15. }
```

Output:

```
Black
```

Overloading In Object-Oriented Programming (OOPs)

Introduction

In object-oriented programming (OOP), overloading is a powerful feature that allows a class to have multiple methods with the same name but different parameters. It enables developers to write more concise and flexible code, enhancing the overall readability and maintainability of the program. This article will delve into the concept of overloading, its types, use cases, and best practices to help you leverage this essential OOP concept in your coding journey.

Understanding the Basics of Overloading

What is Overloading?

Overloading in Object Oriented Programming is a fundamental concept in object-oriented programming, allowing a class to define multiple methods with the same name but varying parameters. This technique enables developers to perform different operations based on

the type and number of arguments provided, streamlining code organization and enhancing its flexibility.

Why Use Overloading in OOP?

Overloading provides several benefits, including cleaner code, reduced redundancy, and improved readability. It allows developers to create intuitive and descriptive method names, making the code more self-explanatory and easier to maintain.

Method Overloading

Defining Method Overloading

In method overloading, a class can have multiple methods with the same name but different parameters. When a method is called, the appropriate version is automatically selected based on the provided arguments.

Overloading by Changing Number of Parameters

Method overloading allows us to create methods that accept a different number of parameters, catering to various use cases without cluttering the codebase.

Overloading by Using Different Data Types

Another aspect of method overloading involves defining methods that accept different data types as arguments. This versatility allows developers to handle a wide range of data inputs effectively.

Rules and Limitations of Method Overloading

While overloading is a valuable tool, it is essential to follow certain rules and be aware of its limitations to avoid potential issues and maintain code clarity.

Constructor Overloading

Exploring Constructor Overloading

Just like method overloading, constructor overloading enables classes to have multiple constructors, each accepting different parameters, facilitating object initialization based on various scenarios.

Using Different Constructors in a Class

By providing multiple constructors, developers can create objects with different initial states, tailoring them to specific needs.

Operator Overloading

Introduction to Operator Overloading

Operator overloading allows classes to define custom behaviors for standard operators, such as +, -, *, /, and more, when applied to class objects.

Overloading Arithmetic Operators

By overloading arithmetic operators, developers can implement custom mathematical operations for user-defined objects.

Overloading Comparison Operators

Comparison operator overloading enables developers to define how objects of a class are compared, providing more meaningful results for complex data structures.

Customizing Behavior with Operator Overloading

Operator overloading can significantly improve code readability and expressiveness when dealing with custom objects.

Function Overloading vs. Function Overriding

Key Differences Between Overloading and Overriding

While both overloading and overriding involve methods with the same name, they serve distinct purposes and are used in different contexts.

When to Use Overloading and Overriding

Understanding the appropriate scenarios for using overloading and overriding is crucial for writing efficient and maintainable code.

Overloading in Inheritance

Extending Overloading to Inherited Classes

Inheritance in OOP allows developers to create derived classes that inherit properties and behaviors from base classes, including overloaded methods.

Inheritance in C++

The capability of a [class](#) to derive properties and characteristics from another class is called **Inheritance**. Inheritance is one of the most important features of Object-Oriented Programming.

Inheritance is a feature or a process in which, new classes are created from the existing classes. The new class created is called “derived class” or “child class” and the existing class is known as the “base class” or “parent class”. The derived class now is said to be inherited from the base class.

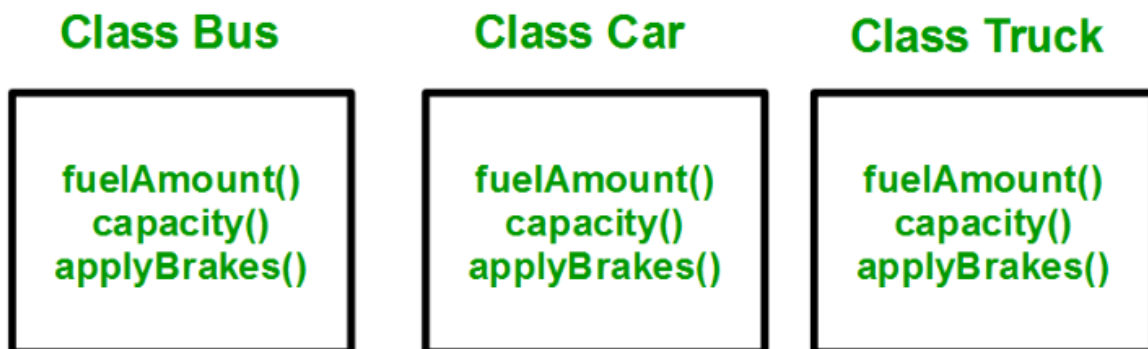
When we say derived class inherits the base class, it means, the derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own. These new features in the derived class will not affect the base class. The derived class is the specialized class for the base class.

- **Sub Class:** The class that inherits properties from another class is called Subclass or Derived Class.
- **Super Class:** The class whose properties are inherited by a subclass is called Base Class or Superclass.

The article is divided into the following subtopics:

Why and when to use inheritance?

Consider a group of vehicles. You need to create classes for Bus, Car, and Truck. The methods `fuelAmount()`, `capacity()`, `applyBrakes()` will be the same for all three classes. If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes as shown below figure:



Types Of Inheritance:-

1. Single inheritance
2. Multilevel inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

Types of Inheritance in C++

1. Single Inheritance: In single inheritance, a class is allowed to inherit from only one class. i.e. one subclass is inherited by one base class only.

```
class subclass_name : access_mode base_class
{
    // body of subclass
};
OR
class A
{
    ... ..
};
class B: public A
{
    ... ..
};
```

Multiple Inheritance: Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class**.

Syntax:

```
class subclass_name : access_mode base_class1, access_mode
base_class2, ....
{
    // body of subclass
};
class B
{
    ... ..
};
class C
{
    ... ..
};
class A: public B, public C
{
    ... ..
};
```

Multilevel Inheritance: In this type of inheritance, a derived class is created from another derived class.

Syntax:-

```
class C
{
    ... ..
```

```
};
class B:public C
{
... ..
};
class A: public B
{
... ..
};
```

Hierarchical Inheritance: In this type of inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.

Syntax:-

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

Hybrid (Virtual) Inheritance: Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance.

A special case of hybrid inheritance: Multipath inheritance:

A derived class with two base classes and these two base classes have one common base class is called multipath inheritance. Ambiguity can arise in this type of inheritance.

What is type parameter in programming?

In computer programming languages, `TypeParameter` is a generic label used in generic programming to reference an unknown data type, data structure, or class.

`TypeParameter` is most frequently used in C++ templates and Java generics .

`TypeParameter` is similar to a metasyntactic variable (e.g., `foo` and `bar`), but distinct.

Java Interface Generic Parameter

We can develop more adaptable and reusable interfaces in Java since an interface can include a generic type parameter. Classes, methods, and various other interfaces which may operate on a wide range of distinct data types can be defined using generic interfaces.

Declaring an arbitrary interface follows a similar syntax to doing so for a generic class. The type parameter is enclosed in angle brackets ("`<`" and "`>`") and can be any acceptable Java identifier.

1. **public interface** `MyInterface<T>` {
2. **void** `doSomething(T value)`;
3. }

The interface "`MyInterface`" in this illustration has a type argument "`T`" that accepts any type. The interface specifies a single method named "`doSomething`" that accepts a type "`T`" parameter.

The generic interface can be used to build classes that implement it for various data types. For instance:

1. **public class** `MyClass` **implements** `MyInterface<String>` {
2. **public void** `doSomething(String value)` {
3. `System.out.println("Doing something with " + value)`;
4. }
5. }
6. **public class** `AnotherClass` **implements** `MyInterface<Integer>` {
7. **public void** `doSomething(Integer value)` {
8. `System.out.println("Doing something with " + value)`;
9. }
10. }

The generic interface "`MyInterface`" is implemented by the two classes "`MyClass`" and "`AnotherClass`" in this example. While the second class utilises the integer data type, the first class utilises the string data type.

There are various advantages to using generic interfaces. It enables us to create more flexible and reusable code. We can limit the amount of code duplication by defining interfaces that interact with a range of different data types.

It also increases the type safety of our code, which is an advantage. Generic interfaces enable us to guarantee that the types given to the methods match the anticipated data types, reducing the chance of runtime problems.

Finally, Java interface generic attributes offer an effective tool for developing adaptable and reusable interfaces. We may create classes, methods, as well as additional interfaces that can

operate on a wide range of various data types by using generic interfaces. This can lessen code duplication and raise the general level of code quality. A generic interface in Java may have a number of type parameters, which may be used to define the interface's methods, variables, and other features. Angle brackets ">" are used to define the type parameters after the interface name.

Here is an example program that utilises a standard user interface and enables input of values of various data types. After performing several operations on the input values using the generic interface, the program presents the results.

Data Abstraction & ADT

When you start your car, you don't need to know the intricate workings of the starter motor. All you need to do is turn the key to initiate the sequence. If successful, the engine will turn over. This real-world example highlights the programming concept of **data abstraction**, which allows a programmer to protect/hide the implementation of a process and only gives the keys to other functions or users. You only need to know enough about a given function to run it but don't need to know (or care) about how the internal code works.

Stack Data Type Example

In this example, we're creating an object-oriented C++ program that creates an abstract data type in the form of a **stack**, in which items can be pushed onto the top and popped off the top. The program simulates your browsing history. Each page you visit is stored in a stack. As you click 'back' on the browser, you are removing/viewing the top element of the stack. As you go forward you are pushing them back onto the stack.

This example also shows a little different method for declaring functions from a class. Notice how they're declared inside the class but created outside of it. Stacks are very common, so let's see how they can work for us.

Abstraction

In object oriented programming (OOP), **abstraction** and **data hiding** are two important concepts. Abstraction is a process of extracting important information without involving the complete detail of the system. On the other hand, data hiding is a process of wrapping the data in a single unit, i.e., to achieve data encapsulation.

What is Abstraction?

Abstraction, or **data abstraction**, is one of the object-oriented methodologies. Abstraction is defined as the process of hiding the internal implementation and keeping the complicated procedures hidden from the user. Thus, abstraction only displays the required services or parts of the system. This is usually achieved using the '**abstract**' class concept, and by implementing **interfaces**.

Types of Abstraction

There are three types of abstraction, namely,

- **Procedural abstraction** – Procedures are written as methods/functions one followed by the other. It is done in sequence in order to obtain abstraction.
- **Data Abstraction** – Abstraction is achieved using data set that is used to describe the object of the class.
- **Control Abstraction** – Here, it is achieved by using objects by ensuring that the details of the object are hidden from user.

Abstraction helps in securing the software. It does not affect endusers, since the developers can perform changes internally. Abstraction ensures that the application is flexible and user-friendly.

What is Data Hiding?

In OOP, **data hiding** refers to the task of hiding the internal data from outsiders. With the help of data hiding, the internal data would not be accessible by irrelevant people. Data hiding can be achieved using access specifiers, such as '**private**', and '**protected**'. It acts like a security layer as it ensures that users cannot access internal data without authentication.

In data hiding, **getters** and **setters** can be used to access the data or to modify it. "getters" help in accessing the private data, while "setters" help in changing the settings of data, i.e., modifying the private data.

Difference between Abstraction and Data Hiding

S.No.	Abstraction	Data Hiding
1.	The process of hiding the implantation details and showing only the functionality of program is referred to as abstraction.	The process of hiding important and sensitive data from any unintended access is known as data hiding.
2.	Abstraction helps in extracting relevant information from a larger set of information.	Data hiding helps to increase safety by preventing outside attacks.
3.	Abstraction focuses on reducing the complexity of code.	The purpose of data hiding is to achieve data encapsulation.
4.	Abstraction solves the design level problems.	Data hiding solves the implementation level problems.
5.	There are three types of data abstractions namely, procedural	Data hiding is not divided into subtypes.

abstraction, data abstraction, and control abstraction.

Data abstraction refers to providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.

Data abstraction is a programming (and design) technique that relies on the separation of interface and implementation.

Let's take one real life example of a TV, which you can turn on and off, change the channel, adjust the volume, and add external components such as speakers, VCRs, and DVD players, BUT you do not know its internal details, that is, you do not know how it receives signals over the air or through a cable, how it translates them, and finally displays them on the screen.

Visibility Modes in C++

Introduction:

One of the key components of *object-oriented programming* in C++ is *data hiding*, which enables us to conceal internal object characteristics, such as *data members*, and prohibits program functions from directly accessing an *object's internal representation*, *data members*, and *member functions*.

Access modifiers define the limitations on access to *class member functions*. In C++, *visibility modes*, such as *private*, *public*, and *protected*, are also referred to as *access modifiers*. In the following sections of this article, we will examine each of them in further detail.

We can determine how the derived class will inherit the *base class* and which *base class members* of the derived class will be able to access by using visibility modes. Since they are used to specify the accessibility and visibility level of class members, they are known as *visibility mode* or *access specifier*.

When the *derived class objects* can utilize a member function in the base class, *access specifiers* are mostly employed in inheritance. The following is the standard syntax for utilizing an access specifier when a child class is derived from the base class:

1. `class` Base {
- 2.
3. `// base class content`
4. `};`
- 5.

```

6. class Derived: access_specifier Base {
7.
8. // child's class content
9. };

```

In the code above, the **access_specifier** tells us which base class members will be obtained and made accessible by the **derived class**. There is also a base class called **Base** and a derived class called **Derived**.

Multiple public, protected, or private parts are permissible in C++ classes. The code below demonstrates how visibility modes can be used in a class:

```

1. class base {
2.
3. public:
4. // public members place
5.
6. protected:
7. // protected members place
8.
9. private:
10. // private members place
11. };

```

The Private Members:

When a **child class** inherits from its **parent class** in the private visibility mode, all of the base class's members-**public**, **private**, and **protected members**-become **private** in the **derived class**.

The only functions that have **access** to these members outside of the derived class are the **member functions** of the derived class or the friend functions.

When the derived class access mode is set to **private**, the following block diagram can demonstrate how data members of the base class are inherited.



The **private variable** is only available by **member functions** within the base class itself, as you can see from the image above because it is of the **private type** within the **base class**. As a result, it is not accessible in the **derived class**. The protected **variable b** and the **public variable c** of the base class will change to private in the derived class, and only its member functions will have access to them.

A very simple real-life example will assist us in understanding when each **access specifier** is necessary. Let's say you want to update your status on a social networking app, but you have some criteria and limitations on who can see this status online. Therefore, if you want this status to be hidden from others or to be visible "**only for me**", you are adhering to the private **access specifier** notion.

Accessibility:

The private mode's accessibility can be summed up as follows:

Accessibility	Private mode
Same class	Yes
Derived class	No
Other class	No
Anywhere in the program	No

The **members specified** by the **private access specifier** are exclusively reachable inside the same class; they are not reachable within any **derived classes** or other classes in the program.

The Public Members:

The **public, private, and protected members** of the base class remain **public, private, and protected members** of the derived class when we inherit a child class from the parent class in **public visibility mode**. All members of the **parent class** remain accessible in the **public visibility mode**.

The **child's class** and all other classes can access the **parent's public members**. Only the **derived class** and its **inherited classes** have access to the protected base class members. Although, the **child class** cannot access the **private members**.

When the **derived class** access mode is set to **public**, the following block diagram can demonstrate how base class data members are inherited.



Accessibility:

The public mode's accessibility can be summed up as follows:

Accessibility	Private mode
Same class	Yes
Derived class	Yes
Other class	Yes
Anywhere in the program	Yes

Users have access to the members defined under the public **access specifier** everywhere within the same class, derived classes, in any other class, or even outside the class.

The Protected Members:

When a **child class** is derived from its **parent class** in the protected **visibility mode**, all of the base class's members are turned into protected members of the derived class. As a result, only the **derived class** and its member functions can now access these members. The **inherited subclasses** of this derived class will have access to these members, which are likewise inheritable.

When the **derived class access mode** is set to **protected**, the following block diagram can demonstrate how data members of the base class are inherited.



As you can see from the image above, the **private variable** is of the **private type** and is, therefore, only accessible by the **base class member functions**. As a result, it is not available in the **derived class**. Only the **inherited subclasses** of this **derived class** will have access to the **public variable a**, which will become protected in the **derived class**. The **base class's protected >variable b** will continue to be protected in the **derived class** and will only be available within the **derived class** and its **inherited classes**.

Using the real-world example of updating your status on a social media app we used earlier; let's say you want your status to only be visible to your friends and friends. In this case, you are adhering to the "**Protected access specifier**" concept.

Example:

Let's look at a case study to better grasp the protected visibility mode notion and the limitations it imposes:

```

1. class Parent
2. {
3.     private:
4.         int x1;
5.
6.     protected:
7.         int x2;
8.
9.     public:
10.        int x3;
11. };
12. class Child: protected Parent
13. {
14.
15. };

```

The **public variable x3** will also be **inherited** from the **parent class** as the protected member and will be accessible in the **Child class**, but the **private variable x1** will not be. Both the **protected variable x2** and the **public variable x3** will be inherited from the **Parent class** and will be accessible in the **Child class**.

Accessibility:

The protected mode's accessibility can be summed up as follows:

Accessibility	Private mode
Same class	Yes
Derived class	Yes
Inherited subclasses of the derived class	Yes
Other class	No
Anywhere in the program	No

Differences between Private, Public, and Protected in C++:

Private	Public	Protected
Only the functions inside the base class can access class members that have been designated as private.	The participants of the class that have been made public are accessible from anywhere.	The base class and its derived classes allow access to class members that have been designated as protected.
The keyword private is used to declare private members, followed by the colon (:) symbol .	The keyword public is used to declare members, followed by the colon (:) symbol.	Members that are protected are identified by using the word protected and the colon (:) character.
Private members are not passed down across classes.	In a class, public members may be inherited.	In a class, protected members can be inherited.
It offers its data members the highest level of protection.	It offers no security at all for the data it holds.	Compared to private access specifier, it offers less security.
A friend function can be used to access private members.	Access to public members is possible both with and without the aid of the friend feature.	Protected members are inaccessible outside the class and cannot be accessed by the friend function, but they may be accessed by any subclass (derived class) of that class.

Modules

A *module* is a set of source code files that are compiled independently of the source files (or more precisely, the [translation units](#) that import them). Modules eliminate or reduce many of the problems associated with the use of header files. They often reduce compilation times. Macros, preprocessor directives, and nonexported names declared in a module aren't visible outside the module. They have no effect on the compilation of the translation unit that imports the module. You can import modules in any order without concern for macro redefinitions. Declarations in the importing translation unit don't participate in overload resolution or name lookup in the imported module. After a module is compiled once, the results are stored in a binary file that describes all the exported types, functions, and templates. The compiler can process that file much faster than a header file. And, the compiler can reuse it every place where the module is imported in a project.

C++ Classes and Objects

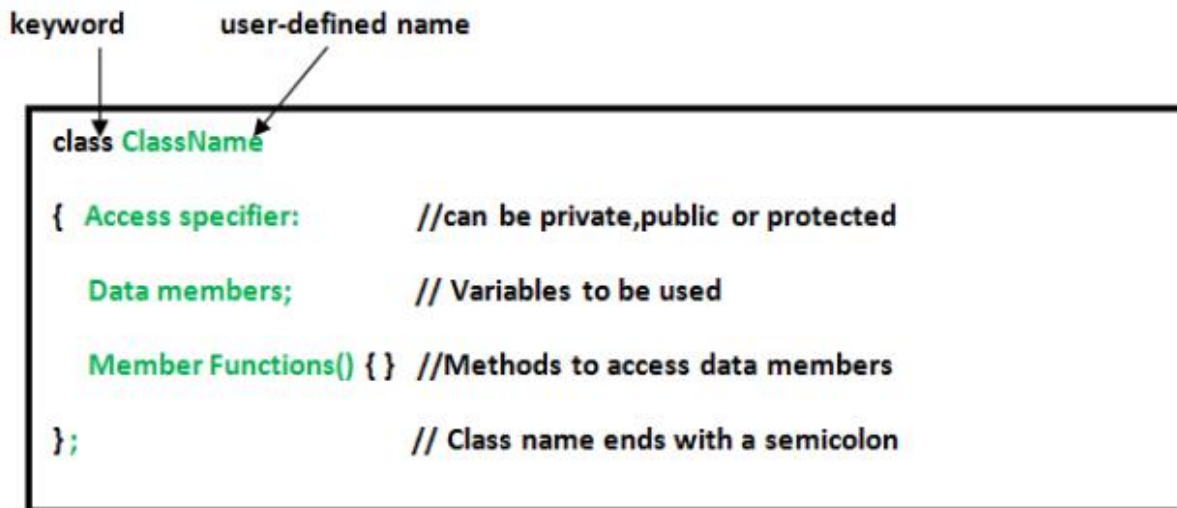
Class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object. For Example: Consider the Class of **Cars**. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 *wheels*, *Speed Limit*, *Mileage range*, etc. So here, Car is the class, and wheels, speed limits, and mileage are their properties.

- A Class is a user-defined data type that has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables together, these data members and member functions define the properties and behavior of the objects in a Class.
- In the above example of class *Car*, the data member will be *speed limit*, *mileage*, etc, and member functions can be *applying brakes*, *increasing speed*, etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using the keyword `class` followed by the name of the class. The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.



Declaring Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Accessing data members and member functions: The data members and member functions of the class can be accessed using the dot('.') operator with the object. For example, if the name of the object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()*.

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member. This access control is given by [Access modifiers in C++](#). There are three access modifiers: **public**, **private**, and **protected**.

Member Functions in Classes

There are 2 ways to define a member function:

- Inside class definition
- Outside class definition

To define a member function outside the class definition we have to use the **scope resolution:: operator** along with the class name and function name.

C++ Object Oriented Programming

Being an object-oriented programming language, C++ uses objects to model real-world problems

Unlike procedural programming, where functions are written to perform operations on data, OOP involves creating objects that contain both data and functions.

An object has two characteristics: **attributes** and **behavior**. For example, a car can be an object. And, it has

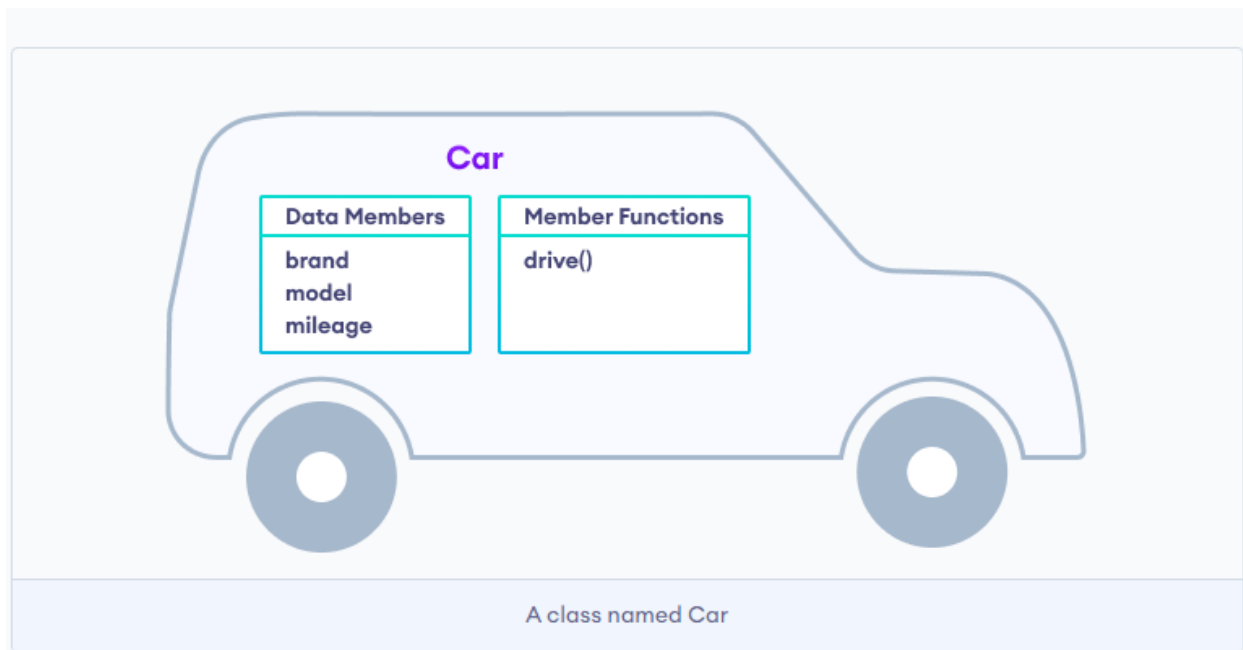
- **attributes** - brand, model, size, mileage, etc.
- **behavior** - driving, acceleration, parking, etc.

C++ Class

A **class** is a blueprint for the **object**.

We can think of a class as the technical design (prototype) of a car. It contains all the details about the brand, model, mileage, etc. We can then build different cars based on these descriptions. Here, each distinct car is an object.

An example for this can be:



```
class Car {  
    public:  
  
    // class data  
    string brand, model;  
    int mileage = 0;  
  
    // class function  
    void drive(int distance) {  
        mileage += distance;  
    }  
};
```

In the above code, we have used the `class` keyword to create a class named `Car`. Here,

- `brand` and `model` are class attributes used to store data
- `drive()` is a class function used to perform some operation

The `public` keyword represents an **access modifier**. To learn more, visit [C++ Access Modifiers](#).

C++ Objects

An object is an instance of a class.

For example, the `car` class defines the model, brand, and mileage. Now, based on the definition, we can create objects like

```
Car suv;  
Car sedan;  
Car van;
```

Here, `suv`, `sedan`, and `van` are objects of the `car` class. Hence, the basic syntax for creating objects is:

Basic Principles of C++ Object-Oriented Programming

The foundational principles of C++ OOP are:

Encapsulation	Bundling of related data and functions together within a single entity.
Abstraction	Showing only the essential attributes of the class, while hiding the technical details from the user.
Inheritance	Using features from an existing class within a new class, without modifying the existing class.
Polymorphism	Ability of the same entity (function or operator) to behave differently in different scenarios.

1. C++ Encapsulation

In C++, object-oriented programming allows us to bundle together data members (such as variables, arrays, etc.) and its related functions into a single entity. This programming feature is known as encapsulation.

. C++ Abstraction

In object-oriented programming, abstraction refers to the concept of showing only the necessary information to the user i.e. hiding the complex details of program implementation and execution.

C++ Inheritance

Inheritance in C++ allows us to create a new class (derived class) from an existing class (base class).

The derived class inherits features from the base class and can have additional features of its own.

```

#include <iostream>
using namespace std;

// base class
class Vehicle {
public:

    string brand;

    void show_brand() {
        cout << "Brand: " << brand << endl;
    }
};

// derived class
class Car : public Vehicle {
public:

    string model;

    void show_model() {
        cout << "Model: " << model << endl;
    }
};

int main() {

    // create an object of Car class
    Car my_car;

    // initialize variables of my_car
    my_car.brand = "Honda";
    my_car.model = "Accord";

    // display variables of my_car
    my_car.show_brand();
    my_car.show_model();

    return 0;
}

```

Output

```
Brand: Honda
```

Model: Accord

Here,

- `Vehicle` is the base class.
- `Car` is the derived class.

The derived class inherits the features of the base class. We can see this from the `brand` variable and the `show_brand()` function, since the `Car` object `my_car` can access them.

In addition to the features of the base class, the derived class also has features of its own. The unique features of the `Car` class are:

- `model` - a string variable
- `show_model()` - a function that prints the `model` variable.

C++ Polymorphism

Polymorphism is the ability to use a common function (or operator) in multiple ways.

In C++, polymorphism is implemented with the help of [function overloading](#), [operator overloading](#), [function overriding](#), and [virtual functions](#).